17.7 The Full Flow: CI to GitOps

Topic 7 of 7 | ~5 hours | Difficulty: Advanced

1 What it is

This topic joins the pieces into one pipeline: **Developer -> Git -> CI -> Docker image -> GitOps repo -> Argo CD -> Kubernetes**.

- **CI (continuous integration)** builds, tests and scans the code and publishes an immutable container image to a registry.
- **CD (continuous delivery) becomes GitOps:** instead of CI running `kubectl apply`, CI's last step *edits the GitOps repo* (bumps an image tag, via direct commit or pull request). The in-cluster agent does the deploy.
- **Secrets** are the hard part: the desired state lives in Git, but passwords and tokens must not. Three well-known approaches keep Git safe: **Sealed Secrets** (encrypt with a cluster key, commit the ciphertext), **SOPS** (encrypt values in the file with age/KMS keys, decrypt at apply time) and **External Secrets Operator (ESO)** (commit only a *reference*; the operator fetches the value from AWS Secrets Manager, Vault, etc.).

2 Why it matters in DevOps

- A split of responsibilities - CI proves the artifact is good, GitOps decides where it runs - gives the cleanest audit trail and the smallest credential footprint.
- Getting the hand-off wrong causes loops (a manifest commit retriggers CI), races (two pipelines bump the same file) and un-deployable releases (the image tag in Git does not exist).
- A leaked secret in Git history is permanent in practice; rotating it is mandatory. Choosing a secrets strategy before the first commit is much cheaper than after. See Secrets Management and Encryption §11.4 and Secrets Scanning §11.11.
- This flow is what most "platform engineering" golden paths automate for product teams (see Platform Engineering §0.13).

3 Real-world system example

The pattern of **separate app and config repositories**, CI that pushes an image and then updates a manifest repo, and Argo CD deploying the result is the standard flow shown in Argo CD's documentation and in countless public engineering write-ups; the Argo CD "Best Practices" page explicitly recommends a separate config repository. **Sealed Secrets** (from Bitnami) is widely used in small and mid-size clusters; **SOPS** (originally from Mozilla, now a CNCF sandbox project) is supported natively by Flux's kustomize-controller; **External Secrets Operator** is a CNCF project that syncs from AWS Secrets Manager, Google Secret Manager, Azure Key Vault, HashiCorp Vault and others. Kubernetes `Secret` objects are only base64-encoded, not encrypted, which is why none of these approaches commit them as-is - see Kubernetes Secrets §8.7.

4 Diagram

End-to-end flow:

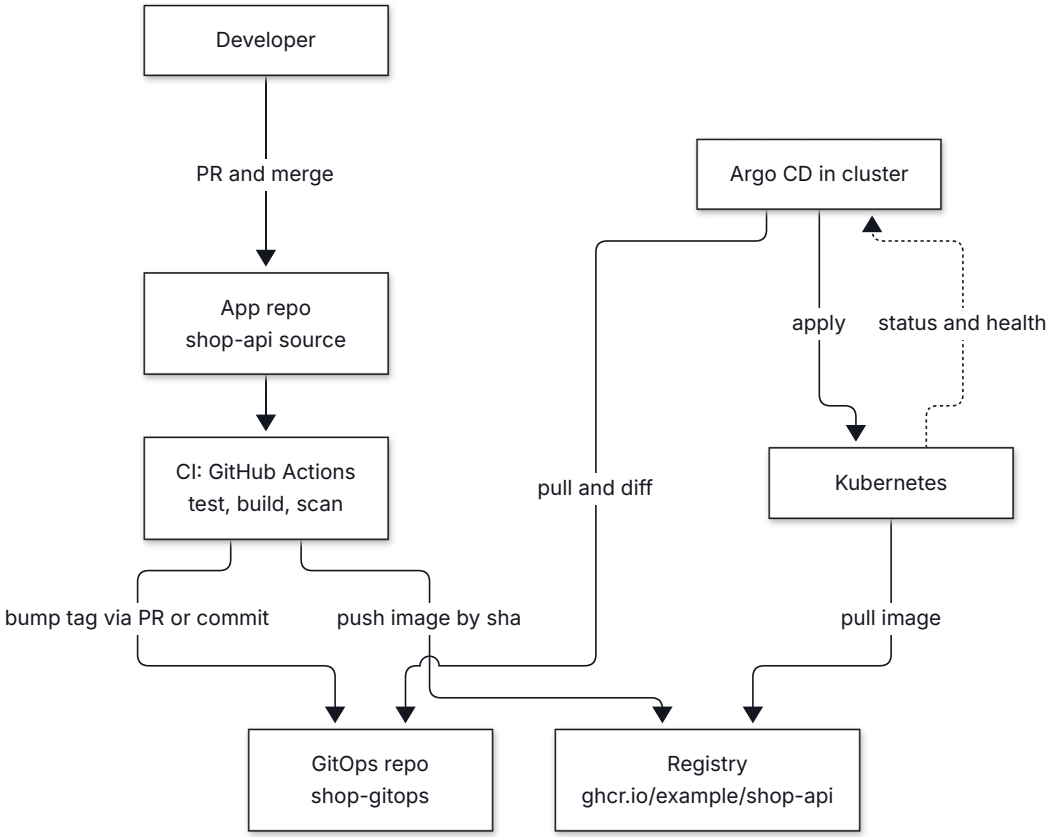


Figure 17.7-1

Three ways to keep secrets out of plain text in Git:

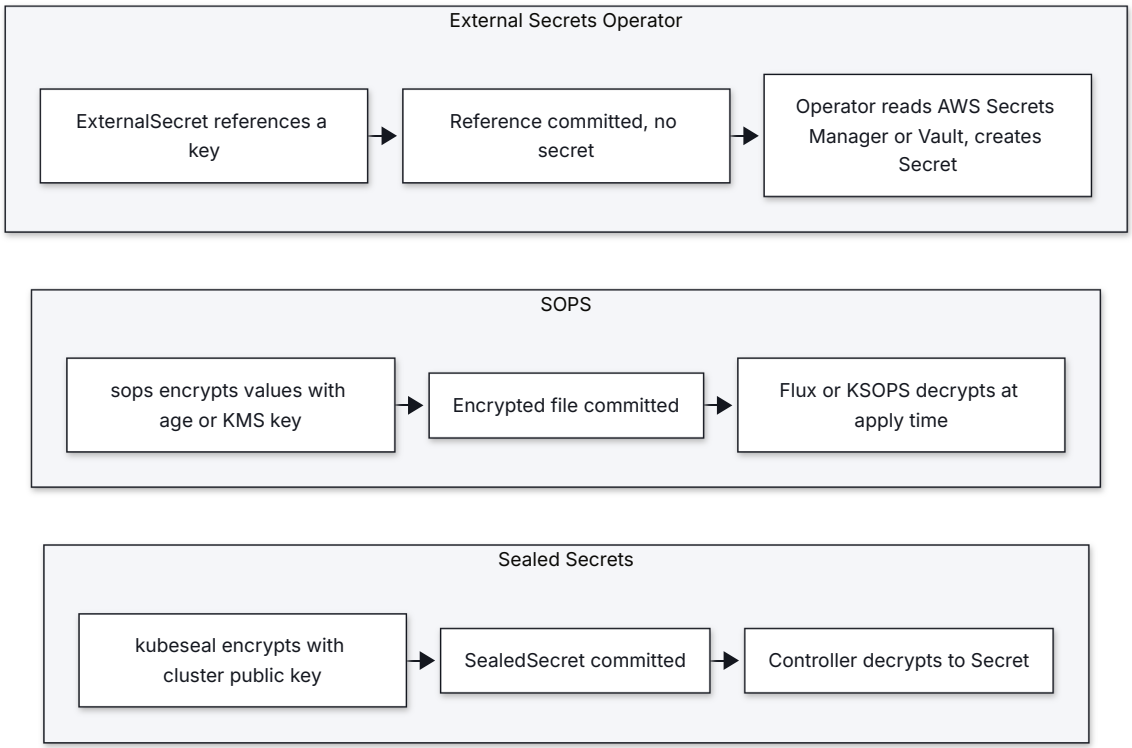


Figure 17.7-2

5 How it works

The CI side

1. Developer merges to `main` in the **app repo**.
2. CI runs tests (Automated Testing §5.3), builds the image, scans it (Image Scanning §11.9) and pushes it tagged with the **commit SHA** (and a semantic version for releases). SHA tags are unique and traceable.
3. The final CI job updates the **GitOps repo**: the dev overlay's tag, either by direct commit (dev only) or a PR (staging, prod). This job uses a token scoped to the GitOps repo only. The cluster never sees a CI credential.
4. Argo CD syncs. Optionally CI waits for `argocd app wait` or polls the Application status, then runs end-to-end tests.

Loop prevention: the GitOps repo has no build pipeline that triggers on manifest changes (or it uses path filters), so a tag bump does not retrigger an image build.

Secrets: choosing an approach

	Sealed Secrets	SOPS	External Secrets Operator
What is in Git	`SealedSecret` (ciphertext)	the Secret file with encrypted values	`ExternalSecret` (a reference only)
Source of truth for the value	Git (ciphertext)	Git (ciphertext)	external secret manager (AWS, Vault, ...)
Key management	controller's private key in the cluster (back it up!)	age key or cloud KMS key	operator's credentials to the store
Argo CD support	works natively (controller does the decrypt)	needs a plugin such as KSOPS or helm-secrets	works natively
Flux support	works natively	built into kustomize-controller	works natively
Rotation	re-seal and commit	re-encrypt and commit	rotate in the store, operator syncs (`refreshInterval`)
Best for	small teams, simple setups	teams wanting encrypted-in-Git with flexible keys	orgs with an existing secret manager and audit needs

Whichever you choose: Kubernetes does not encrypt Secrets at rest in etcd by default, so enable encryption at rest and restrict RBAC on Secrets (RBAC in Depth §16.1).

Example manifests (illustrative, for orientation; all values are placeholders):

YAML

```yaml
# Sealed Secrets: produced by kubeseal, safe to commit
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: shop-api-db
  namespace: shop-dev
spec:
  encryptedData:
    DATABASE_URL: AgBy3i4OJSWK...   # long ciphertext produced by kubeseal
  template:
    metadata:
      name: shop-api-db
      namespace: shop-dev
```

```yaml
# .sops.yaml: encrypt only the data of Secret manifests with an age public key
creation_rules:
  - path_regex: .*secret.*\.yaml$
    encrypted_regex: ^(data|stringData)$
    age: age1examplepublickeyexamplepublickeyexamplepublickeyexamplepub
```

```yaml
# External Secrets Operator: a reference to a value held elsewhere
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: shop-api-db
  namespace: shop-dev
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: SecretStore
    name: aws-secrets
  target:
    name: shop-api-db
  data:
    - secretKey: DATABASE_URL
      remoteRef:
        key: shop/dev/database-url
```

(ESO's API version was `external-secrets.io/v1beta1` in older releases; check your installed CRDs with `kubectl get crd externalsecrets.external-secrets.io -o yaml`.)

6 Step-by-step guide

Prerequisites: kind cluster with Argo CD (topic 2 §17.2), the `shop-gitops` repo with a dev Application, a GitHub account, `helm`, `kubectl`. Use a public image (e.g. `nginx`) if you do not build one. Cost: none (GitHub Actions free tier).

Part A: the CI hand-off

1. **Create a fine-grained token for the GitOps repo.** In GitHub: Settings -> Developer settings -> Fine-grained tokens, access to only `shop-gitops`, permission "Contents: read and write". Store it as an Actions secret named `GITOPS_TOKEN` in the **app** repo.
2. **Add the CI workflow to the app repo.**

```yaml
# .github/workflows/ci.yaml in shop-api
name: ci
```

```yaml
on:
  push:
    branches: [main]
permissions:
  contents: read
  packages: write
jobs:
  build-and-bump:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${{ github.repository_owner }}/shop-api:${{ github.sha }}
      - name: Bump dev tag in GitOps repo
        env:
          GITOPS_TOKEN: ${{ secrets.GITOPS_TOKEN }}
          IMAGE_TAG: ${{ github.sha }}
          OWNER: ${{ github.repository_owner }}
        run: |
          git clone "https://x-access-token:${GITOPS_TOKEN}@github.com/${OWNER}/shop-
  gitops.git"
          cd shop-gitops/apps/shop-api/overlays/dev
          kustomize edit set image "ghcr.io/${OWNER}/shop-api=ghcr.io/${OWNER}/shop-
  api:${IMAGE_TAG}"
          cd ../../../..
          git config user.name "ci-bot"
          git config user.email "ci-bot@example.com"
          git commit -am "ci: shop-api dev → ${IMAGE_TAG}"
          git push
```

Staging and prod follow the PR-based promotion from Environment Promotion §17.4. Expected result: a green run, a new image in GitHub Packages and a commit by `ci-bot` in `shop-gitops`; Argo CD syncs dev within minutes. (Tag, action and runner versions evolve; check current releases.)

3. **Watch the hand-off end to end.**

```bash
kubectl -n shop-dev get deploy shop-api -o
  jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Expect the new SHA tag. Make a trivial code change, push to `main`, and repeat.

Part B: secrets with Sealed Secrets (hands-on) and ESO (sandbox)

4. **Install the Sealed Secrets controller and CLI.**

```bash
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets
helm repo update
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system
kubectl -n kube-system rollout status deploy/sealed-secrets
```

   Install `kubeseal` from the project's releases page (or your package manager). The Deployment name can differ by chart version; check `kubectl -n kube-system get deploy | grep sealed`.

5. **Seal a secret and commit only the ciphertext.**

```bash
kubectl create secret generic shop-api-db -n shop-dev \
  --from-literal=DATABASE_URL='postgres://shop:CHANGE_ME@10.0.0.5:5432/shop' \
  --dry-run=client -o yaml \
| kubeseal --controller-name sealed-secrets --controller-namespace kube-system --
  format yaml \
> apps/shop-api/overlays/dev/sealed-db.yaml
head -12 apps/shop-api/overlays/dev/sealed-db.yaml
```

   Expect `kind: SealedSecret` and a long `encryptedData` value. Add the file under `resources:` in the dev overlay, commit, push. Argo CD applies it and the controller creates a normal `Secret`:

```bash
kubectl -n shop-dev get secret shop-api-db
```

   Back up the controller's sealing key (`kubectl -n kube-system get secret -l sealedsecrets.bitnami.com/sealed-secrets-key -o yaml > key-backup.yaml`, store it outside Git and the cluster); lose it and every SealedSecret is unrecoverable on a rebuilt cluster.

6. **Reference the secret from the Deployment.** In the base Deployment container, add:

```yaml
envFrom:
  - secretRef:
      name: shop-api-db
```

Commit, push, and confirm with `kubectl -n shop-dev exec deploy/shop-api -- printenv DATABASE_URL | cut -c1-12` (shows only the prefix).

7. **ESO in the sandbox (no cloud needed) using the `fake` provider.**

```bash
helm repo add external-secrets https://charts.external-secrets.io
helm install external-secrets external-secrets/external-secrets -n external-secrets --
  create-namespace --set installCRDs=true
```

```yaml
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: fake
  namespace: shop-dev
spec:
  provider:
    fake:
      data:
        - key: shop/dev/api-key
          value: not-a-real-key
---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: shop-api-key
  namespace: shop-dev
spec:
  refreshInterval: 1m
  secretStoreRef:
    kind: SecretStore
    name: fake
  target:
    name: shop-api-key
  data:
    - secretKey: API_KEY
      remoteRef:
        key: shop/dev/api-key
```

Apply, then `kubectl -n shop-dev get externalsecret,secret shop-api-key`. Expect `SecretSynced` / `Ready`. In production you would swap the `fake` provider for `aws` (Secrets Manager, see Secrets Manager §6.16) using IRSA or similar - never static keys. (If `v1` is not served by your ESO version, use `external-secrets.io/v1beta1`.)

8. **SOPS (read-through).** `age-keygen -o key.txt`, put the public key in `.sops.yaml`, run `sops --encrypt --in-place secret.yaml`; with Flux, set `decryption: {provider: sops, secretRef: {name: sops-age}}` on the Kustomization. With Argo CD use KSOPS or helm-

secrets.

7 Real-world scenario

Illustrative scenario. A developer pastes the production database URL into `values-prod.yaml` "just to test" and merges it. The secret scanner (see Secrets Scanning §11.11) flags it within a minute; the team rotates the password immediately because the value is in Git history forever, then adopts External Secrets Operator so that the GitOps repo contains only references. A month later, a CI-token compromise attempt on the app repo can only write image tags, because the GitOps repo requires PRs with code-owner approval for prod, and the cluster credentials were never in CI. **Lesson:** the flow's value is in the *boundaries*: CI cannot reach the cluster, Git cannot hold secrets, prod needs a human.

8 Common mistakes and best practices

Don't

- Let CI run `kubectl apply` against prod "just for this one service".
- Commit a plain Kubernetes `Secret`, even base64-encoded, or rely on a private repo for secrecy.
- Tag images `latest` or overwrite tags; Argo CD will not notice a changed image behind the same tag.
- Put the CI that builds images on the GitOps repo's manifest changes (infinite loop).
- Lose the Sealed Secrets private key, or the SOPS age key, without a backup.

Do

- Tag images with the commit SHA; promote the same image through environments.
- Use a least-privilege, repo-scoped token (or a GitHub App) for the CI -> GitOps hand-off.
- Choose one secrets strategy per platform, document it and rotate regularly.
- Verify deploys end to end: have CI (or a PostSync hook) smoke-test the new version and report status.
- Sign images and verify them with an admission policy when you are ready (see Admission Controllers §16.6).

9 Tasks

- [ ] **T1 — Wire CI to the GitOps repo** *(≈60 min, intermediate)*
  - **Goal:** a push to the app repo ends with a new image running in dev.

- **Steps:**
  1. Create the token and secret, add the workflow from Part A.
  2. Push a code change to `main`.
  3. Watch Actions, the `ci-bot` commit, and the Argo CD sync.
- **Done when:** the image in `shop-dev` equals the commit SHA of your latest push.

☐ **T2 — Seal a secret** *(≈45 min, intermediate)*

- **Goal:** commit a secret safely.
- **Steps:**
  1. Install Sealed Secrets and `kubeseal`.
  2. Seal `DATABASE_URL`, add it to the dev overlay and push.
  3. Mount it with `envFrom`.
- **Done when:** the Secret exists in `shop-dev`, and `grep -r 'CHANGE_ME' .` in the repo finds nothing.

☐ **T3 — Try External Secrets** *(≈45 min, intermediate)*

- **Goal:** use a reference instead of a ciphertext.
- **Steps:**
  1. Install ESO and apply the `fake` provider example.
  2. Change the value in the SecretStore and wait for `refreshInterval`.
  3. Observe the Secret update.
- **Done when:** `kubectl get secret shop-api-key -o jsonpath='{.data.API_KEY}' | base64 -d` shows the new value.

☐ **T4 — Break it: lose the key** *(≈30 min, advanced)*

- **Goal:** feel why backing up the sealing key matters.
- **Steps:**
  1. Back up the sealing key, then delete the controller's key Secret and restart the controller (a new key is generated).
  2. Observe the old SealedSecret fail to decrypt (`kubectl -n shop-dev describe sealedsecret ...`).
  3. Restore the key backup and restart the controller.
- **Done when:** the SealedSecret decrypts again after restoring the backup.

☐ **T5 — Promote through all stages with a gate** *(≈75 min, advanced)*

- **Goal:** the full flow dev -> staging -> prod.

- **Steps:**
  1. Extend CI to open a PR for staging after the dev deploy is healthy (`argocd app wait` or a smoke test).
  2. Require CODEOWNERS approval for prod.
  3. Promote one release all the way to prod, then roll it back with `git revert` (Rollback Through Git §17.6).
- **Done when:** the commit history of the GitOps repo shows the promotion chain and the revert, all by PR.

☐ **T6 — Prove it: trace a change** *(≈30 min, advanced)*

- **Goal:** narrate the full flow.
- **Steps:**
  1. Without notes, draw Developer -> Git -> CI -> image -> GitOps repo -> Argo CD -> Kubernetes, marking which credentials each hop holds.
  2. Choose a secrets approach for a team using AWS and justify it in three sentences.
- **Done when:** you can state, for each credential, what damage its compromise allows and why it is the smallest possible.

10 Self-check questions

1. Why should CI edit a GitOps repo rather than run `kubectl apply`?
2. Why tag images with the commit SHA?
3. What do you commit with Sealed Secrets, SOPS and ESO respectively?
4. What is the biggest operational risk of Sealed Secrets?
5. How do you stop a manifest commit from retriggering the image build?

ANSWERS

1. No cluster credentials in CI, a reviewable audit trail, drift correction by the in-cluster agent, and a single source of truth.
2. SHAs are unique and immutable, traceable to source, and a changed tag triggers a visible Git diff that Argo CD syncs.
3. A `SealedSecret` ciphertext; a file with SOPS-encrypted values; an `ExternalSecret` reference with no secret value.
4. Losing the controller's private key (no backup) makes all SealedSecrets undecryptable on a rebuilt cluster.

5. Keep the GitOps repo separate from the app repo (or use path filters), so manifest commits do not trigger the app's build pipeline.

11 Further reading

- Argo CD documentation - "Best Practices" (separate config repository) and "Secret Management".
- Sealed Secrets (github.com/bitnami-labs/sealed-secrets), SOPS (getsops.io) and External Secrets Operator (external-secrets.io) documentation.
- Flux documentation - "Manage Kubernetes secrets with SOPS".
- *Continuous Delivery* by Jez Humble and David Farley; see also CI/CD Pipeline Design §5.1.